

# COMPUTER ORGANIZATION

## Unit-II: Register Transfer Language

Comprehensive Study Notes

HPU BCA Semester III

# 1. Register Transfer Language (RTL)

## 1.1 What is Register Transfer Language?

**Register Transfer Language (RTL)** is a symbolic notation used to describe the transfer of data between registers and the operations performed on data in digital systems. It serves as a bridge between the hardware and software descriptions of a computer system.

**Definition:** RTL is a notation for specifying the sequence of micro-operations that transfer data between registers and perform arithmetic and logic operations.

## 1.2 Basic Symbols in RTL

| Symbol                     | Meaning            | Example            |
|----------------------------|--------------------|--------------------|
| <b>R</b>                   | Register           | R1, R2, AC, PC     |
| $\leftarrow$               | Transfer           | $R1 \leftarrow R2$ |
| <b>[ ]</b>                 | Contents           | [R1]               |
| <b>:</b>                   | Assignment/Control | $R1 \leftarrow R2$ |
| <b>+</b>                   | Addition           | $R1 + R2$          |
| <b>−</b>                   | Subtraction        | $R1 - R2$          |
| <b><math>\wedge</math></b> | AND                | $R1 \wedge R2$     |
| <b>'</b>                   | Complement         | $R1'$              |
| <b><math>\vee</math></b>   | OR                 | $R1 \vee R2$       |
| <b><math>\oplus</math></b> | XOR                | $R1 \oplus R2$     |

## 1.3 Register Transfer

**Register transfer** is the movement of data from one register to another.

RTL Notation:  $R2 \leftarrow R1$

**Meaning:** Transfer the contents of register R1 to register R2. (R1 remains unchanged; it is a copy operation.)

## 1.4 Basic Register Transfer Operations

| Operation                   | RTL Notation                           | Description                     |
|-----------------------------|----------------------------------------|---------------------------------|
| <b>Simple Transfer</b>      | $R2 \leftarrow R1$                     | Copy R1 to R2                   |
| <b>Conditional Transfer</b> | If (T) $R2 \leftarrow R1$              | Transfer if condition T is true |
| <b>Bus Transfer</b>         | $BUS \leftarrow R1, R2 \leftarrow BUS$ | Transfer via bus                |
| <b>Memory Transfer</b>      | $M[AR] \leftarrow R1$                  | Transfer to memory              |
| <b>Immediate Transfer</b>   | $R1 \leftarrow 0$                      | Clear register                  |

## 2. Bus and Memory Transfer

### 2.1 What is a Bus?

A **bus** is a set of common lines (wires) that connect multiple registers and memory units. It provides a shared communication path for transferring data.

```

Register A -----|
                  |
Register B -----|----> [ COMMON BUS ] ----> Destination
                  |
Register C -----|
  
```

### 2.2 Three-State Bus Buffers

**Three-state buffers** are used to connect multiple registers to a common bus.

| State              | Description                     |
|--------------------|---------------------------------|
| 0                  | Output is logic 0               |
| 1                  | Output is logic 1               |
| High Impedance (Z) | Output is disconnected from bus |

### 2.3 Bus Transfer using Three-State Buffers

```

BUS ← R1  (Enable R1 on bus)
R2 ← BUS  (Copy bus to R2)
  
```

### 2.4 Memory Transfer

**Memory transfer** involves moving data between registers and memory.

Memory Read:  $MDR \leftarrow M[MAR]$   
 Memory Write:  $M[MAR] \leftarrow MDR$

| Register      | Full Form               | Function                             |
|---------------|-------------------------|--------------------------------------|
| <b>MAR</b>    | Memory Address Register | Holds the address of memory location |
| <b>MDR</b>    | Memory Data Register    | Holds the data to be read/written    |
| <b>M[MAR]</b> | Memory location         | Location pointed by MAR              |

## 2.5 Multiplexed Bus System

A multiplexed bus uses **multiplexers** to select one of many register outputs to connect to the bus, instead of three-state buffers.

```

R0 ---|
R1 ---|----> [ MUX ] ---> BUS ----> R2
R2 ---|
R3 ---|
  
```

## 3. Arithmetic Micro-operations

### 3.1 What are Arithmetic Micro-operations?

**Arithmetic micro-operations** are operations that perform arithmetic calculations on data stored in registers.

```

+-----+
|  ADDITION          |
|  R3 ← R1 + R2      |
+-----+

+-----+
|  SUBTRACTION        |
|  R3 ← R1 - R2      |
+-----+

+-----+
|  INCREMENT          |
|  R1 ← R1 + 1        |
+-----+

+-----+
|  DECREMENT          |
|  R1 ← R1 - 1        |
+-----+

```

### 3.2 List of Arithmetic Micro-operations

| Operation                  | RTL Notation            | Description                      |
|----------------------------|-------------------------|----------------------------------|
| <b>Addition</b>            | $R3 \leftarrow R1 + R2$ | Add R1 and R2, store in R3       |
| <b>Subtraction</b>         | $R3 \leftarrow R1 - R2$ | Subtract R2 from R1, store in R3 |
| <b>Increment</b>           | $R1 \leftarrow R1 + 1$  | Add 1 to R1                      |
| <b>Decrement</b>           | $R1 \leftarrow R1 - 1$  | Subtract 1 from R1               |
| <b>Negation (2's comp)</b> | $R1 \leftarrow -R1$     | Take 2's complement of R1        |

### 3.3 Binary Adder

---

A **Binary Adder** is a circuit that performs addition of binary numbers. The RTL notation is:

$$R3 \leftarrow R1 + R2$$

### 3.4 Binary Adder-Subtractor

---

A **Binary Adder-Subtractor** can perform both addition and subtraction by using XOR gates and a control signal (M).

- If  $M = 0$ : Addition ( $R3 = R1 + R2$ )
- If  $M = 1$ : Subtraction ( $R3 = R1 - R2$ )

### 3.5 Binary Incrementer / Decrementer

---

- **Incrementer:** Adds 1 to a register using a half-adder circuit
- **Decrementer:** Subtracts 1 from a register using a complement-and-add approach

### 3.6 Arithmetic Circuit Operations

---

| Select | Operation   | Result         |
|--------|-------------|----------------|
| 00     | Addition    | $R3 = R1 + R2$ |
| 01     | Subtraction | $R3 = R1 - R2$ |
| 10     | Increment   | $R1 = R1 + 1$  |
| 11     | Decrement   | $R1 = R1 - 1$  |

## 4. Logic Micro-operations

### 4.1 What are Logic Micro-operations?

**Logic micro-operations** perform bitwise logical operations on data in registers.

```

+-----+
|  AND ( $\wedge$ )      |
|  R3  $\leftarrow$  R1  $\wedge$  R2 |
+-----+

+-----+
|  OR ( $\vee$ )        |
|  R3  $\leftarrow$  R1  $\vee$  R2 |
+-----+

+-----+
|  XOR ( $\oplus$ )      |
|  R3  $\leftarrow$  R1  $\oplus$  R2 |
+-----+

+-----+
|  COMPLEMENT (')   |
|  R2  $\leftarrow$  R1'   |
+-----+

```

### 4.2 List of Logic Micro-operations

| Operation         | Symbol     | RTL Notation                   | Description              |
|-------------------|------------|--------------------------------|--------------------------|
| <b>AND</b>        | $\wedge$   | $R3 \leftarrow R1 \wedge R2$   | Bitwise AND              |
| <b>OR</b>         | $\vee$     | $R3 \leftarrow R1 \vee R2$     | Bitwise OR               |
| <b>XOR</b>        | $\oplus$   | $R3 \leftarrow R1 \oplus R2$   | Bitwise XOR              |
| <b>Complement</b> | '          | $R2 \leftarrow R1'$            | Bitwise complement (NOT) |
| <b>NAND</b>       | $\uparrow$ | $R3 \leftarrow R1 \uparrow R2$ | Bitwise NAND             |

|             |              |                                  |              |
|-------------|--------------|----------------------------------|--------------|
| <b>NOR</b>  | $\downarrow$ | $R3 \leftarrow R1 \downarrow R2$ | Bitwise NOR  |
| <b>XNOR</b> | $\odot$      | $R3 \leftarrow R1 \odot R2$      | Bitwise XNOR |

### 4.3 Truth Table for Logic Operations

| A | B | AND | OR | XOR | NAND | NOR | XNOR |
|---|---|-----|----|-----|------|-----|------|
| 0 | 0 | 0   | 0  | 0   | 1    | 1   | 1    |
| 0 | 1 | 0   | 1  | 1   | 1    | 0   | 0    |
| 1 | 0 | 0   | 1  | 1   | 1    | 0   | 0    |
| 1 | 1 | 1   | 1  | 0   | 0    | 0   | 1    |

### 4.4 Applications of Logic Micro-operations

| Operation  | Application                           |
|------------|---------------------------------------|
| AND        | Masking (selective clear bits)        |
| OR         | Setting (selective set bits)          |
| XOR        | Complementing (selective toggle bits) |
| Complement | Bitwise inversion                     |

## 5. Shift Micro-operations

### 5.1 What are Shift Micro-operations?

**Shift micro-operations** move the bits of a register left or right.

```

+-----+
| LOGICAL SHIFT      |
| Left (<<) / Right (>>) |
| Zero fill          |
+-----+

+-----+
| ARITHMETIC SHIFT   |
| Left / Right       |
| Sign bit maintained |
+-----+

+-----+
| CIRCULAR SHIFT     |
| (Rotate)           |
| Bits wrap around   |
+-----+

```

### 5.2 Types of Shifts

#### 5.2.1 Logical Shift

| Shift                      | Description                   | Example (1010) |
|----------------------------|-------------------------------|----------------|
| <b>Logical Left Shift</b>  | Shift bits left, fill with 0  | 1010 → 0100    |
| <b>Logical Right Shift</b> | Shift bits right, fill with 0 | 1010 → 0101    |

Logical Left Shift:  $R1 \leftarrow R1 \ll 1$   
 Logical Right Shift:  $R1 \leftarrow R1 \gg 1$

#### Example: Logical Left Shift

Before: R1 = 1010  
 After: R1 = 0100 (shift left by 1, 0 filled at LSB)

### Example: Logical Right Shift

Before: R1 = 1010  
 After: R1 = 0101 (shift right by 1, 0 filled at MSB)

## 5.2.2 Arithmetic Shift

| Shift                         | Description                         | Example (1010) |
|-------------------------------|-------------------------------------|----------------|
| <b>Arithmetic Left Shift</b>  | Shift bits left, fill with 0        | 1010 → 0100    |
| <b>Arithmetic Right Shift</b> | Shift bits right, maintain sign bit | 1010 → 1101    |

**Arithmetic Right Shift** preserves the sign bit (MSB) by replicating it.

Before: R1 = 1010 (MSB = 1, negative in sign-mag)  
 After: R1 = 1101 (sign bit 1 replicated)

## 5.2.3 Circular Shift (Rotate)

| Shift               | Description                   | Example (1010) |
|---------------------|-------------------------------|----------------|
| <b>Rotate Left</b>  | Shift left, MSB wraps to LSB  | 1010 → 0101    |
| <b>Rotate Right</b> | Shift right, LSB wraps to MSB | 1010 → 0101    |

Example: Rotate Left  
 Before: R1 = 1010  
 After: R1 = 0101 (MSB 1 wraps to LSB)

Example: Rotate Right  
 Before: R1 = 1010  
 After: R1 = 0101 (LSB 0 wraps to MSB)

## 5.3 RTL Notation Summary for Shifts

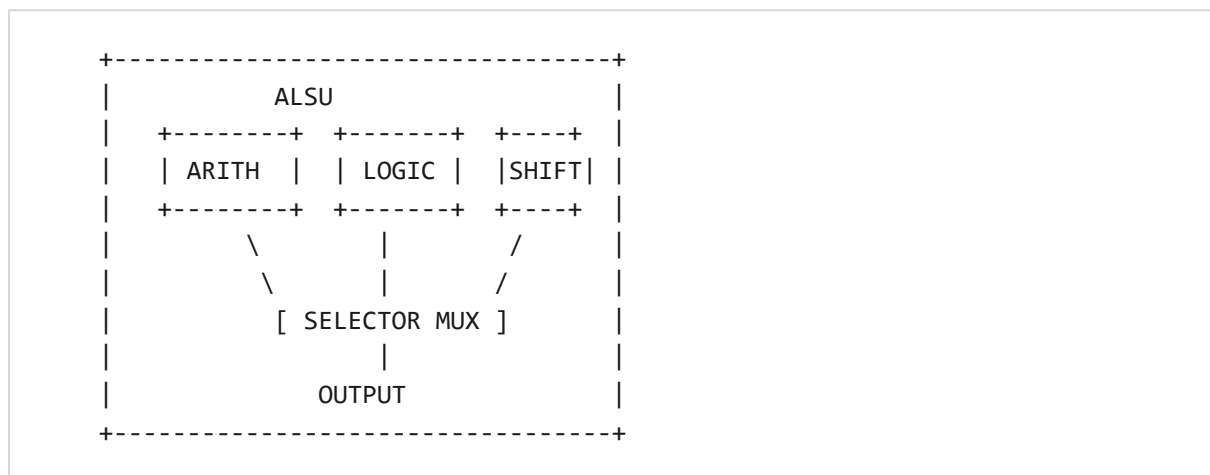
---

| Operation              | RTL Notation                    |
|------------------------|---------------------------------|
| Logical Shift Left     | $R1 \leftarrow \text{shl}(R1)$  |
| Logical Shift Right    | $R1 \leftarrow \text{shr}(R1)$  |
| Arithmetic Shift Left  | $R1 \leftarrow \text{ashl}(R1)$ |
| Arithmetic Shift Right | $R1 \leftarrow \text{ashr}(R1)$ |
| Rotate Left            | $R1 \leftarrow \text{rl}(R1)$   |
| Rotate Right           | $R1 \leftarrow \text{rr}(R1)$   |

## 6. Arithmetic Logic Shift Unit (ALSU)

### 6.1 What is an ALSU?

An **Arithmetic Logic Shift Unit (ALSU)** is a combinational circuit that performs arithmetic, logic, and shift operations on data stored in registers.



### 6.2 Operations of ALSU

| Select | Operation    | Type       |
|--------|--------------|------------|
| 000    | $A + B$      | Arithmetic |
| 001    | $A - B$      | Arithmetic |
| 010    | $A + 1$      | Arithmetic |
| 011    | $A - 1$      | Arithmetic |
| 100    | $A \wedge B$ | Logic      |
| 101    | $A \vee B$   | Logic      |
| 110    | $A \oplus B$ | Logic      |
| 111    | Shift        | Shift      |

## 6.3 Example: ALSU Operations

Inputs: A = 1010 (10), B = 0101 (5)

1. Addition:  $A + B = 1010 + 0101 = 1111$  (15)
2. Subtraction:  $A - B = 1010 - 0101 = 0101$  (5)
3. AND:  $A \& B = 1010 \& 0101 = 0000$  (0)
4. OR:  $A | B = 1010 | 0101 = 1111$  (15)
5. XOR:  $A \wedge B = 1010 \wedge 0101 = 1111$  (15)
6. Shift Left:  $A \ll 1 = 10100$  (20)
7. Shift Right:  $A \gg 1 = 0101$  (5)

## 7. Control of Micro-operations

### 7.1 Control Unit

The **control unit** generates timing signals that determine the sequence in which micro-operations are executed. It ensures that operations occur in the correct order.

### 7.2 Timing and Control

- **Clock cycles** define the timing of micro-operations
- Each clock cycle enables a specific set of micro-operations
- A **sequence counter** or **control counter** keeps track of the current micro-operation step

Clock Cycle 1: T1:  $MAR \leftarrow PC$   
Clock Cycle 2: T2:  $MDR \leftarrow M[MAR]$ ,  $PC \leftarrow PC + 1$   
Clock Cycle 3: T3:  $IR \leftarrow MDR$

### 7.3 Example: Instruction Cycle Control

| Step | Micro-operation                                  | Description                        |
|------|--------------------------------------------------|------------------------------------|
| T0   | $MAR \leftarrow PC$                              | Transfer PC address to MAR         |
| T1   | $MDR \leftarrow M[MAR]$ , $PC \leftarrow PC + 1$ | Read memory; increment PC          |
| T2   | $IR \leftarrow MDR$                              | Transfer instruction to IR         |
| T3   | Decode IR                                        | Decode instruction in control unit |

### 7.4 Conditional Control

Micro-operations can be controlled by **condition flags** such as:

| Flag                | Description                           |
|---------------------|---------------------------------------|
| <b>Z (Zero)</b>     | Set when result is zero               |
| <b>C (Carry)</b>    | Set when addition produces a carry    |
| <b>S (Sign)</b>     | Set when result is negative (MSB = 1) |
| <b>V (Overflow)</b> | Set when arithmetic overflow occurs   |

If (Z)  $PC \leftarrow PC + \text{Offset}$  (Conditional branch)  
If (C)  $R1 \leftarrow R1 + 1$  (Add carry)

# Questions

---

## Short Answer Questions (2–5 Marks)

1. What is Register Transfer Language (RTL)?
2. What are micro-operations?
3. What is a bus transfer?
4. What are arithmetic micro-operations?
5. What are logic micro-operations?
6. What are shift micro-operations?
7. What is a three-state buffer?
8. What is the difference between logical shift and arithmetic shift?
9. What is a binary adder-subtractor?
10. What is an Arithmetic Logic Shift Unit (ALU)?

## Long Answer Questions (10 Marks)

1. Explain Register Transfer Language (RTL) and register transfer operations with examples.
2. Explain bus and memory transfer with three-state buffers.
3. Explain arithmetic micro-operations with examples and hardware implementation.
4. Explain logic micro-operations with truth tables and applications.
5. Explain shift micro-operations with examples of logical, arithmetic, and circular shifts.
6. Explain the Arithmetic Logic Shift Unit (ALU) with its operation table.
7. Explain binary adder, binary adder-subtractor, and binary incrementer circuits.
8. Explain the hardware implementation of shift micro-operations.
9. Explain the control of micro-operations with timing and condition flags.

## Objective Questions (1 Mark)

1. RTL stands for:  
(a) Register Transfer Language    (b) Random Transfer Logic

- (c) Both (d) None
2. Three-state buffer has:  
(a) 0 state (b) 1 state (c) High impedance (d) All of the above
3. The notation  $R2 \leftarrow R1$  means:  
(a) Transfer R1 to R2 (b) Transfer R2 to R1  
(c) Both (d) None
4. Logical shift fills with:  
(a) Sign bit (b) Zero (c) One (d) None
5. Arithmetic right shift preserves:  
(a) LSB (b) Sign bit (c) Carry (d) Zero

# Quick Revision

## RTL Operations Summary

Register Transfer:  $R2 \leftarrow R1$   
Memory Transfer:  $MDR \leftarrow M[MAR]$   
Arithmetic:  $R3 \leftarrow R1 + R2$   
Logic:  $R3 \leftarrow R1 \& R2$   
Shift:  $R1 \leftarrow R1 \ll 1$

## Bus Transfer Summary

$BUS \leftarrow R1$  (Enable R1 on bus)  
 $R2 \leftarrow BUS$  (Copy bus to R2)  
 $M[MAR] \leftarrow MDR$  (Memory write)  
 $MDR \leftarrow M[MAR]$  (Memory read)

## Micro-operation Types

| Type       | Operations                                                   |
|------------|--------------------------------------------------------------|
| Arithmetic | ADD, SUB, INC, DEC, NEG                                      |
| Logic      | AND, OR, XOR, NOT, NAND, NOR, XNOR                           |
| Shift      | Logical Left/Right, Arithmetic Left/Right, Rotate Left/Right |

## Key Formulas

Register Transfer:  $\text{Destination} \leftarrow \text{Source}$

Conditional Transfer: If (T)  $R2 \leftarrow R1$

Memory Read:  $MDR \leftarrow M[MAR]$

Memory Write:  $M[MAR] \leftarrow MDR$

**End of Unit-II Notes**